NYC Urban Mobility Dashboard - Summative Project Plan

- Repository: GitHub - Team4C1 Urban Mobility Data Explorer
- Team Tracker: Team Participation Sheet

1. Technical Stack

Domain	Technologies
Data Engine & ETL	Python (pandas, geopandas, pyarrow)
Backend & REST API	Python (Flask), psycopg2 / SQLAlchemy
Database System	PostgreSQL (Relational schema, Spatial boundaries, Indexes)
Frontend Dashboard	HTML5, CSS3, JavaScript (Vanilla JS / Fetch API, Chart.js, Leaflet.js)

2. Team Roster & Ownership Matrix

Team Members

- Member 1 (Data Engineer & Team Lead):
    - Member Name: _____
    - Tasks: ETL & Data Cleaning
- Member 2 (Database & Algorithm Engineer):
    - Member Name: _____
    - Tasks: PostgreSQL Schema & Custom Algo
- Member 3 (Backend Engineer):
    - Member Name: _____
    - Tasks: REST API development
- Member 4 (Frontend Engineer): [Name]
    - Member Name: _____
    - Tasks: Data Visualisations

a

Detailed Member Responsibilities

Member 1: Data Engineer & Team Lead

- Technical Tasks:
    - Ingest raw datasets (.parquet trip records, taxi_zone_lookup.csv, and taxi_zones.geojson) using Python.
    - Clean records by filtering negative fare amounts, missing coordinates, zero-distance records, and logical outliers.
    - Engineer 3 derived features (e.g., avg_speed_mph, tip_percentage, fare_per_mile).
    - Maintain and document a structured exclusion log for rejected records.
- Administrative Tasks:
    - Manage and update the official Team Participation Sheet (ensure Lead has Edit access and Members have Comment-only access).
- Report Deliverable: Author Section 1: Problem Framing & Dataset Analysis.

Member 2: Database & Algorithm Engineer

- Technical Tasks:
    - Write SQL DDL scripts to create normalized PostgreSQL tables (trips, zones, spatial_boundaries) with proper primary keys, foreign keys, and indexes.
    - Write automated ingestion scripts to populate PostgreSQL from cleaned Python exports.
    - Implement a manual custom algorithm in pure Python (e.g., custom QuickSort/MergeSort or sliding-window anomaly detector) without relying on built-in functions (sort(), heapq, Pandas .sort_values(), or SQL ORDER BY).
- Report Deliverable: Author Section 3: Algorithmic Logic, Pseudo-code & $O(N)$ Complexity Analysis.

Member 3: Backend Engineer

- Technical Tasks:
    - Develop REST API endpoints using Flask returning structured JSON payloads (/api/trips, /api/zones, /api/insights).
    - Connect Flask to PostgreSQL for efficient parameter filtering (date ranges, fares, locations).
    - Expose Member 2's custom algorithm through a dedicated API route.
- Report Deliverable: Design the full system architecture diagram and author Section 2: System Architecture & Design Decisions.

Member 4: Frontend Engineer

- Technical Tasks:
    - Build a responsive web dashboard using HTML5, CSS3, and JavaScript (Fetch API).
    - Implement dynamic UI controls for user filtering (date, location, fare amount, trip distance).
    - Render interactive visualizations using Chart.js (analytics charts) and Leaflet.js (GeoJSON spatial maps).
- Report Deliverable: Author Section 4: 3 Urban Mobility Insights & Visualizations.

All Members (Joint Deliverables)

- Author Section 5: Reflection & Future Work of the PDF report.
- Record the 5-minute video walkthrough (Architecture, Custom Algorithm, Dashboard Demo).
- Prepare a clean, fully runnable README.md file with installation steps (pip install, DB setup, launch instructions).

3. Deliverables Checklist

- Codebase Archive (.zip) & GitHub Repository: Fully runnable code with meaningful commit history.
- Database Dump / Schema Files: Complete .sql scripts for PostgreSQL table creation and data import.
- PDF Technical Documentation Report (2–3 Pages): Covering Sections 1 through 5.
- Project README.md: Clear, step-by-step launch and configuration instructions.
- 5-Minute Video Walkthrough: Demonstrating architecture, algorithm execution, and working UI features.
- Team Participation Sheet: Fully completed with correct permission settings.

4. Compliance, Guardrails & Integrity Rules

1. Zero-Grade Failures: Submissions missing any of the core deliverables (Video, Documentation Report, or README.md) or containing non-executable code will automatically receive a grade of zero.
2. AI & Code Vibe Policy: Maximum allowable AI-generated code is 20% (minimum 80% manual code). AI tools are strictly limited to assisting with formatting/writing the README.md. Generic or AI-generated filler in the documentation report will incur heavy penalties.

3. Custom Algorithm Rules: The custom algorithm must be implemented from scratch using pure Python control structures. Do not use sort(), heapq, Pandas .sort_values(), or SQL ORDER BY.

4. Sheet Access Controls: The Team Lead must be the sole editor of the Team Participation Sheet. All other team members must be granted Comment-only permissions.

5. Link Verification: Ensure all submitted links (GitHub, Team Tracker Sheet, Video Walkthrough) are publicly accessible prior to submission. Broken links will not be graded.